Cosmos EVM 최적화 실험 결과

개요

이 문서는 Phase 3에서 제안한 최적화들의 예상 실험 결과와 분석 방법론을 정리한다. 실제 구현 전에 예상 결과를 문서화하여 실험 설계의 타당성을 검증하고, 향후 실제 실험 시 비교 기준을 제공한다.

관련 문서:

- 01-cosmos-evm-profiling-analysis.md - 성능 병목 분석
- 02-optimization-implementation-plan.md - 최적화 구현 계획
- 03-consensus-experiment-design.md - 합의 실험 설계
- 04-evm-optimization-benchmark.md - 벤치마크 설계

---

1. 실험 환경

1.1 하드웨어 사양

```
+-----------------------------------------+
| 테스트 환경 사양                          |
+-----------------------------------------+
| CPU: Intel Core i9-12900K (16C/24T)      |
| RAM: 64 GB DDR5-4800                      |
| SSD: Samsung 980 Pro 2TB NVMe             |
| Network: 10 Gbps                          |
+-----------------------------------------+
| 클라우드 환경 (네트워크 테스트용)          |
+-----------------------------------------+
| Provider: AWS                             |
| Instance: c6i.4xlarge (16 vCPU, 32 GB)    |
| Region: us-east-1                         |
| Network: 10 Gbps                          |
+-----------------------------------------+
```

1.2 소프트웨어 환경

```yaml
# 테스트 환경
go_version: "1.21.6"
cosmos_sdk: "v0.50.x"
cosmos_evm: "main branch"
cometbft: "v0.38.x"
os: "Ubuntu 22.04 LTS"

# 벤치마크 도구
benchstat: "latest"
pprof: "go tool pprof"
prometheus: "v2.45.0"
grafana: "v10.0.0"
```

1.3 테스트 시나리오

시나리오	설명	트랜잭션 유형
Simple Transfer	EOA → EOA 전송	단순 전송
ERC20 Transfer	토큰 전송	2개 스토리지 업데이트
Uniswap Swap	DEX 스왑	다중 컨트랙트 호출
Complex DeFi	복합 DeFi 트랜잭션	10+ 스토리지 업데이트
Block Building	100 tx/블록	혼합 워크로드

---

2. Storage Read Cache 실험 결과

2.1 마이크로 벤치마크

2.1.1 캐시 히트 성능

```
BenchmarkStorageReadCacheHit (예상 결과)
=========================================
Implementation     | ns/op   | B/op  | allocs/op
-------------------|---------|-------|----------
Baseline (no cache)| 485     | 256   | 5
With BlockCache    | 48      | 48    | 1
-------------------|---------|-------|----------
Improvement        | -90.1%  | -81.3%| -80.0%
```

분석:

- 캐시 히트 시 KVStore 접근 완전 회피
- 메모리 할당 80% 감소 (GC 부하 감소)
- 예상 개선: **10x 빠른 읽기**

2.1.2 캐시 미스 성능

```
BenchmarkStorageReadCacheMiss (예상 결과)
=========================================
Implementation     | ns/op   | B/op  | allocs/op
-------------------|---------|-------|----------
Baseline           | 1,234   | 512   | 8
With BlockCache    | 1,285   | 560   | 9
-------------------|---------|-------|----------
Overhead           | +4.1%   | +9.4% | +12.5%
```

분석:

- 캐시 미스 시 약 4% 오버헤드 (Bloom filter + LRU 조회)
- 추가 메모리 할당 발생
- 허용 가능한 수준의 오버헤드

2.1.3 혼합 워크로드

```
BenchmarkStorageReadCacheMixed (예상 결과)
==========================================
Hit Rate | Baseline ns/op | With Cache ns/op | Improvement
---------|---------------|------------------|------------
10%      | 1,234          | 1,115            | -9.6%
30%      | 1,234          | 871              | -29.4%
50%      | 1,234          | 627              | -49.2%
70%      | 1,234          | 383              | -69.0%
90%      | 1,234          | 139              | -88.7%
```

분석:

- 히트율 50% 이상에서 유의미한 성능 개선
- DeFi 워크로드는 일반적으로 70%+ 히트율 예상

2.2 블록 레벨 캐시 공유

```
BenchmarkBlockLevelCacheSharing (예상 결과)
===========================================
Scenario: 100 tx/block, Uniswap-like contract

Without Block Cache:
  - Total read time: 123.4 ms
  - Unique slots read: 250
  - Total slot accesses: 5,000
  - Cache hit rate: 0%

With Block Cache:
  - Total read time: 31.5 ms
  - Unique slots read: 250
  - Total slot accesses: 5,000
  - Cache hit rate: 95%
  - Improvement: -74.5%
```

분석:

- 동일 컨트랙트 반복 접근 시 극적인 성능 개선
- Uniswap 풀 상태 (reserves, liquidity) 캐싱 효과
- 블록 내 교차 트랜잭션 캐시 공유 성공

2.3 Bloom Filter 효과

```
BenchmarkBloomFilterEffectiveness (예상 결과)
=============================================
Entries  | False Positive Rate | LRU Lookups Saved
---------|---------------------|-------------------
100      | 0.1%                | 99.9%
1,000    | 0.5%                | 99.5%
10,000   | 1.2%                | 98.8%
100,000  | 2.8%                | 97.2%
```

분석:

- 10,000개 엔트리까지 1% 미만 false positive
- Bloom filter로 불필요한 LRU 조회 97%+ 방지
- 메모리 효율적: 100,000 엔트리에 ~128KB

3. Batched State Commits 실험 결과

3.1 배치 크기별 성능

```
BenchmarkBatchSizeImpact (예상 결과)
====================================
Batch Size | ns/op   | B/op    | allocs/op | vs Baseline
-----------|---------|---------|-----------|------------
1 (현재)   | 45,670  | 45,670  | 1,234     | baseline
10         | 12,340  | 15,450  | 345       | -73.0%
50         | 8,560   | 10,280  | 156       | -81.3%
100        | 7,230   | 8,760   | 89        | -84.2%
500        | 6,890   | 8,120   | 45        | -84.9%
1000       | 6,780   | 7,980   | 34        | -85.2%
```

분석:

- 배치 크기 100에서 최적 성능 도달 (~84% 개선)
- 100 이상에서는 한계 수익 체감
- 권장 배치 크기: **100-500**

3.2 메모리 사용량

```
BenchmarkBatchMemoryUsage (예상 결과)
=====================================
Batch Size | bytes/entry | Total MB | Peak MB
-----------|-------------|----------|--------
100        | 128         | 0.012    | 0.015
500        | 128         | 0.061    | 0.075
1000       | 128         | 0.122    | 0.150
5000       | 128         | 0.610    | 0.750
```

분석:

- 엔트리당 ~128 bytes 고정 오버헤드
- 배치 크기 1000에서도 150KB 미만
- 메모리 제약 없음

3.3 트랜잭션 처리량 영향

```
BenchmarkApplyTransactionWithBatching (예상 결과)
=================================================
Scenario           | Baseline | With Batching | Improvement
-------------------|----------|---------------|------------
Simple Transfer    | 0.8 ms   | 0.7 ms        | -12.5%
ERC20 Transfer     | 1.2 ms   | 0.9 ms        | -25.0%
Uniswap Swap       | 3.5 ms   | 2.4 ms        | -31.4%
Complex DeFi       | 8.2 ms   | 5.1 ms        | -37.8%
```

분석:

- 스토리지 쓰기가 많을수록 개선 효과 증가
- Complex DeFi에서 최대 ~38% 개선
- 단순 전송에서도 12% 개선

---

4. Journal Optimization 실험 결과

4.1 사전 할당 효과

```
BenchmarkJournalAppendPreallocated (예상 결과)
==============================================
Implementation       | ns/op | B/op  | allocs/op
---------------------|-------|-------|----------
Dynamic (현재)       | 234.5 | 8,234 | 167
Preallocated (1024)  | 156.7 | 2,456 | 45
---------------------|-------|-------|----------
Improvement          | -33.2%| -70.2%| -73.1%
```

분석:

- 슬라이스 재할당 오버헤드 제거
- 메모리 할당 70% 감소
- 대부분의 트랜잭션이 1024개 미만의 journal 엔트리 사용

4.2 Object Pool 효과

```
BenchmarkJournalEntryAllocation (예상 결과)
===========================================
Implementation  | ns/op | B/op | allocs/op
----------------|-------|------|----------
New Allocation  | 45.6  | 96   | 1
Pooled          | 12.3  | 0    | 0
----------------|-------|------|----------
Improvement     | -73.0%| -100%| -100%
```

분석:

- sync.Pool로 완전한 할당 제거
- 재사용으로 GC 부하 제거
- 고빈도 호출 경로에서 효과적

4.3 Snapshot/Revert 성능

```
BenchmarkSnapshotRevert (예상 결과)
===================================
Scenario        | Baseline | Optimized | Improvement
----------------|----------|-----------|------------
Shallow (10)    | 1.2 µs   | 0.8 µs    | -33.3%
Deep (1000)     | 45.6 µs  | 28.9 µs   | -36.6%
Nested (10x10)  | 15.3 µs  | 9.2 µs    | -39.9%
```

분석:

- 중첩 스냅샷에서 최대 40% 개선
- Revert 경로에서도 일관된 개선
- Smart contract 실행의 try-catch 패턴에 유리

---

5. 종합 성능 실험

5.1 트랜잭션 유형별 최종 성능

```
Combined Optimizations (예상 결과)
===================================
Transaction Type   | Baseline | All Opts | Improvement
-------------------|----------|----------|------------
Simple Transfer    | 1.567 ms | 1.234 ms | -21.3%
ERC20 Transfer     | 2.345 ms | 1.678 ms | -28.4%
Uniswap Swap       | 4.567 ms | 2.890 ms | -36.7%
Complex DeFi       | 9.876 ms | 5.432 ms | -45.0%
```

분석:

- 모든 최적화 적용 시 평균 30%+ 성능 개선
- 복잡한 트랜잭션일수록 효과 증가
- 목표 (20%+) 초과 달성

5.2 블록 빌드 성능

```
BenchmarkBlockBuilding (예상 결과)
==================================
Tx Count | Baseline | All Opts  | Improvement | TPS
---------|----------|-----------|-------------|--------
10       | 23.4 ms  | 17.2 ms   | -26.5%      | 581
50       | 112.5 ms | 78.9 ms   | -29.9%      | 633
100      | 234.5 ms | 156.7 ms  | -33.2%      | 638
500      | 1,234 ms | 789.5 ms  | -36.0%      | 633
```

분석:

- 블록 크기 증가에 따른 확장성 개선
- 100 tx/block에서 ~640 TPS 달성
- 캐시 효과가 누적되어 대형 블록에서 더 효과적

5.3 메모리 프로파일

```
Memory Usage Comparison (예상 결과)
===================================
Component          | Baseline | Optimized | Delta
-------------------|----------|-----------|-------
StateDB per tx     | 45 KB    | 38 KB     | -15.6%
Block cache        | 0        | 2-5 MB    | +2-5 MB
Batch writer       | 0        | 50 KB     | +50 KB
Journal pool       | 0        | 100 KB    | +100 KB
-------------------|----------|-----------|-------
Net change/block   | 4.5 MB   | 6.0 MB    | +33%
```

분석:

- 블록당 ~1.5 MB 추가 메모리 사용
- CPU 시간 대비 메모리 trade-off 합리적
- 64 GB RAM 기준 충분한 여유

---

6. BLS 서명 합의 실험 (예상)

6.1 서명 집계 성능

```
BenchmarkBLSSignatureAggregation (예상 결과)
============================================
Validator Count | Individual | Aggregated | Improvement
----------------|------------|------------|------------
4               | 1.2 ms     | 0.8 ms     | -33.3%
10              | 3.0 ms     | 0.9 ms     | -70.0%
50              | 15.0 ms    | 1.2 ms     | -92.0%
100             | 30.0 ms    | 1.5 ms     | -95.0%
```

분석:

- 검증자 수 증가에 따른 선형 확장성
- 100 검증자에서 95% 서명 검증 시간 감소
- O(n) → O(1) 검증 복잡도

6.2 메시지 복잡도

```
Message Complexity Comparison (예상 결과)
=========================================
Validator Count | CometBFT O(n²) | HotStuff O(n) | Reduction
----------------|----------------|---------------|----------
4               | 16             | 8             | -50.0%
10              | 100            | 20            | -80.0%
50              | 2,500          | 100           | -96.0%
100             | 10,000         | 200           | -98.0%
```

분석:

- 검증자 100명 기준 98% 메시지 감소
- 네트워크 대역폭 요구사항 대폭 감소
- 대규모 검증자 세트 실용화

6.3 합의 라운드 시간

```
Round Time Comparison (예상 결과)
=================================
Network Latency | CometBFT | With BLS | Improvement
----------------|----------|----------|------------
10 ms           | 850 ms   | 320 ms   | -62.4%
50 ms           | 1,200 ms | 450 ms   | -62.5%
100 ms          | 1,800 ms | 680 ms   | -62.2%
```

분석:

- 네트워크 지연에 관계없이 ~62% 개선
- 3단계 → 2단계 프로토콜 효과
- 블록 finality 시간 대폭 단축

---

7. 회귀 테스트 결과

7.1 정확성 검증

```
Correctness Tests (예상 결과)
============================
Test Category           | Tests | Passed | Coverage
------------------------|-------|--------|----------
StateDB Unit Tests      | 156   | 156    | 92.3%
Integration Tests       | 89    | 89     | 87.5%
E2E Tests               | 45    | 45     | 78.9%
Fuzzing (1M iterations) | N/A   | Pass   | N/A
```

분석:

- 모든 기존 테스트 통과
- 새로운 경로에 대한 추가 테스트 작성
- Fuzzing으로 엣지 케이스 검증

7.2 호환성 검증

```
Compatibility Tests (예상 결과)
===============================
Category              | Status | Notes
----------------------|--------|-------------------
Ethereum JSON-RPC     | Pass   | 100% 호환
ERC20/721/1155        | Pass   | 표준 동작 유지
Uniswap V2/V3         | Pass   | DEX 정상 동작
Aave V3               | Pass   | Lending 정상 동작
EIP-2929/2930         | Pass   | Access list 지원
EIP-1559              | Pass   | Dynamic fee 지원
```

분석:

- 기존 DApp과 완전 호환
- Ethereum 도구 체인 정상 동작
- API 변경 없음

---

8. 프로덕션 권장 사항

8.1 최적화 적용 우선순위

순위	최적화	예상 효과	구현 난이도	위험도
1	Storage Read Cache	+30-40% TPS	중	낮음
2	Batched Commits	+20-30% TPS	낮음	낮음
3	Journal Optimization	+5-10% TPS	낮음	매우 낮음
4	BLS Signatures	+50%+ 확장성	높음	중

8.2 설정 권장값

```toml
# config.toml 권장 설정
[evm.optimization]
# Storage Read Cache
block_cache_enabled = true
block_cache_size = 10000
bloom_filter_size = 131072  # 128KB

# Batched Commits
batch_commit_enabled = true
batch_size = 100

# Journal Optimization
journal_prealloc_size = 1024
use_entry_pool = true

# BLS (실험적)
bls_signatures_enabled = false  # 프로덕션 준비 후 활성화
```

8.3 모니터링 메트릭

```
# Prometheus metrics to monitor
evm_cache_hit_rate          # 목표: > 70%
evm_cache_size_bytes        # 목표: < 100MB
evm_batch_flush_count       # 배치 플러시 빈도
evm_batch_entries_per_flush # 플러시당 엔트리 수
evm_journal_realloc_count   # 재할당 횟수 (목표: 0)
evm_pool_get_count          # 풀 사용량
evm_pool_miss_count         # 풀 미스 (목표: 낮음)
```

9. 향후 작업

9.1 추가 최적화 기회

1. **Parallel Transaction Execution**
   - 독립적인 트랜잭션 병렬 처리
   - 예상 효과: 멀티코어 환경에서 2-4x
2. **State Snapshot Optimization**
   - IAVL 스냅샷 최적화
   - 예상 효과: 블록 커밋 시간 20% 감소
3. **Precompile Caching**
   - 빈번한 precompile 결과 캐싱
   - 예상 효과: BLS/SHA256 등에서 50%+ 개선

9.2 실제 구현 체크리스트

- [ ] Storage Read Cache 구현 및 통합
- [ ] Batched Commits 구현 및 통합
- [ ] Journal Optimization 구현 및 통합
- [ ] 종합 벤치마크 실행
- [ ] 프로덕션 테스트넷 배포
- [ ] 모니터링 대시보드 구성
- [ ] 문서화 및 PR 제출

---

10. 결론

10.1 핵심 성과 요약

지표	기준	목표	예상 결과	달성률
트랜잭션 처리량	1,500 TPS	2,000 TPS	2,100 TPS	140%
블록 빌드 시간	250 ms	200 ms	160 ms	180%
메모리 효율성	45 KB/tx	40 KB/tx	38 KB/tx	117%
캐시 히트율	N/A	70%	80%	114%

10.2 기술적 기여

1. **Storage Read Cache**: 블록 레벨 읽기 캐시로 90%+ 캐시 히트 개선
2. **Batched Commits**: 배치 쓰기로 84% KVStore 오버헤드 감소
3. **Journal Optimization**: 사전 할당 및 풀링으로 70% 메모리 할당 감소
4. **BLS Signatures**: $O(n^2) \rightarrow O(n)$ 메시지 복잡도 달성

10.3 프로젝트 영향

- **Cosmos EVM 생태계**: 고처리량 EVM 체인 실현
- **DeFi 애플리케이션**: 더 낮은 지연시간, 더 높은 처리량
- **검증자 운영**: 하드웨어 요구사항 감소
- **사용자 경험**: 더 빠른 트랜잭션 확인

참고 자료

관련 문서

- 01-cosmos-evm-profiling-analysis.md
- 02-optimization-implementation-plan.md
- 03-consensus-experiment-design.md
- 04-evm-optimization-benchmark.md
- 05-developer-tools.md

외부 자료

- Go Benchmarking
- pprof Documentation
- BLS12-381
- HotStuff Paper

문서 버전: 1.0 **작성일**: 2025-01 **작성자**: B-Harvest Core Developer Candidate **페이지 수**: ~25 페이지

부록: 실험 데이터 시트

A.1 Raw Benchmark Data Template

```
benchmark,baseline_ns,optimized_ns,baseline_b,optimized_b,baseline_allocs,optimized_allocs
BenchmarkStorageReadCacheHit,485,48,256,48,5,1
BenchmarkStorageReadCacheMiss,1234,1285,512,560,8,9
BenchmarkBatchedCommits/100,45670,7230,45670,8760,1234,89
BenchmarkJournalAppend,234.5,156.7,8234,2456,167,45
BenchmarkApplyTransaction,1567000,1234000,45000,38000,890,756
BenchmarkBlockBuilding/100,234500000,156700000,4500000,3800000,89000,75600
```

A.2 프로파일링 데이터 수집 스크립트

```bash
#!/bin/bash
# scripts/collect_benchmark_data.sh

RESULTS_DIR="results/$(date +%Y%m%d_%H%M%S)"
mkdir -p $RESULTS_DIR

# Run benchmarks
go test -bench=. -benchmem -count=10 \
    ./tests/integration/x/vm/... \
    | tee $RESULTS_DIR/benchmark.txt

# CPU profile
go test -bench=BenchmarkApplyTransaction \
    -cpuprofile=$RESULTS_DIR/cpu.prof \
    -benchtime=30s ./tests/integration/x/vm/

# Memory profile
go test -bench=BenchmarkApplyTransaction \
    -memprofile=$RESULTS_DIR/mem.prof \
    -benchtime=30s ./tests/integration/x/vm/

# Generate reports
go tool pprof -pdf $RESULTS_DIR/cpu.prof > $RESULTS_DIR/cpu.pdf
go tool pprof -pdf $RESULTS_DIR/mem.prof > $RESULTS_DIR/mem.pdf

echo "Results saved to $RESULTS_DIR"
```

A.3 결과 분석 스크립트

```python
#!/usr/bin/env python3
# scripts/analyze_results.py

import pandas as pd
import matplotlib.pyplot as plt

def analyze_benchmark_results(baseline_file, optimized_file):
    baseline = pd.read_csv(baseline_file)
    optimized = pd.read_csv(optimized_file)

    # Calculate improvements
    merged = baseline.merge(optimized, on='benchmark', suffixes=('_base', '_opt'))
    merged['ns_improvement'] = (merged['ns_base'] - merged['ns_opt']) / merged['ns_base'] * 100
    merged['mem_improvement'] = (merged['b_base'] - merged['b_opt']) / merged['b_base'] * 100

    # Generate charts
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Time improvement
    axes[0].barh(merged['benchmark'], merged['ns_improvement'])
    axes[0].set_xlabel('Improvement (%)')
    axes[0].set_title('Execution Time Improvement')

    # Memory improvement
    axes[1].barh(merged['benchmark'], merged['mem_improvement'])
    axes[1].set_xlabel('Improvement (%)')
    axes[1].set_title('Memory Usage Improvement')

    plt.tight_layout()
    plt.savefig('benchmark_comparison.png')

    return merged

if __name__ == '__main__':
    results = analyze_benchmark_results('baseline.csv', 'optimized.csv')
    print(results.to_markdown())
```